

软件过程改进方案设计文档

基于《空箱》Unity 项目的软件过程定义，设计 AI 技术融入的改进方案

1. 引言与改进总览

1.1 改进背景

《空箱》项目基于 ISO/IEC 12207、15504、33001 标准定义了 6 个主过程（P1-P6）和 3 个支撑过程（S1-S3）。前期《AI 技术应用场景匹配报告》已识别出 5 个 AI 可显著提效的关键环节。本文档针对其中优先级最高的 **P3（架构与详细设计）** 和 **P4（实现与自测）** 进行深度改进方案设计，同时对 P2、P5/S2、S3 给出简要改进方向。

1.2 改进原则

原则	说明
可落地性	所有方案基于当前可用的真实 AI 工具，避免纯理论构想
人机协同	AI 定位为辅助者与审查者，最终决策权归人类开发者
渐进式融入	在现有流程中嵌入 AI 节点，而非推翻重建
质量优先	所有 AI 输出物须经人工验证方可进入下一阶段

1.3 改进范围

主攻：P3（架构与详细设计）★★★★
P4（实现与自测）★★★★
简要：P2（需求与规则建模）★★☆
P5/S2（测试与质量保证）★★☆
S3（文档与知识沉淀）★☆☆

2. 新增 AI 相关角色定义

为支撑 AI 技术融入，在原有角色体系基础上新增以下角色：

2.1 AI 架构顾问（AI Architecture Advisor）

属性	说明
职责	根据输入的项目约束与需求，生成多套架构方案、UML 图；对人工设计的架构进行评审与模式推荐
工作节点	P3 过程的设计方案生成阶段
工具载体	GPT-5.5 + PlantUML（通过结构化 Prompt 驱动）
输出物	架构方案对比文档、PlantUML 类图/序列图、设计模式建议

属性	说明
与人类分工	AI 负责方案发散与初稿生成，人类负责决策与质量把关

2.2 AI 代码审查员 (AI Code Reviewer)

属性	说明
职责	对每次代码提交进行自动化审查，检测潜在缺陷、性能问题、架构偏离；生成审查意见
工作节点	P4 过程的代码提交后、人工审查前
工具载体	DeepSeek-V4-Pro + Claude Code Agent
输出物	审查报告（按严重程度分类：MUST_FIX / SUGGESTION / NITPICK）
与人类分工	AI 负责首轮全量扫描与模式匹配，人类负责业务逻辑审查与最终判定

2.3 AI 测试分析师 (AI Test Analyst)

属性	说明
职责	从需求描述和代码变更自动生成测试用例，预测高风险模块
工作节点	P5/S2 过程的测试准备阶段
工具载体	LLM + Unity Test Framework
输出物	测试用例集（含边界条件）、缺陷风险热力图

3. P3 架构与详细设计改进方案 (★ 重点)

3.1 当前过程回顾

P3 是《空箱》项目的架构中枢。其当前活动流程如下：

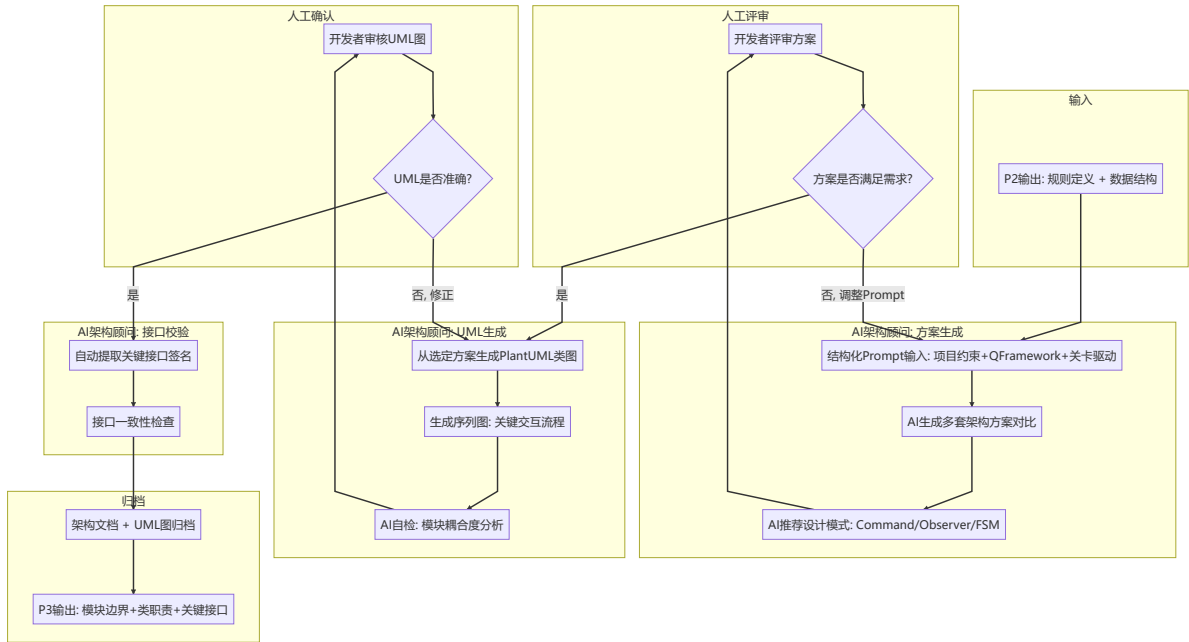


核心痛点：

- 架构方案仅依赖个人经验，缺乏多方案对比
- UML 建模手工绘制，迭代成本高
- 跨模块耦合点识别依赖人工走查，易遗漏

3.2 AI 融入方案

改进后在原有流程中嵌入 4 个 AI 参与节点：



3.3 流程节点详解

节点 1：AI 架构方案生成

输入：结构化的项目约束 Prompt，包含：

- 技术栈约束（Unity 2022.3、C#、QFramework、UniTask、Addressables）
- 架构风格约束（Model/System/Command 分层、事件驱动）
- 核心功能需求（关卡加载、对话推进、信号触发、实体交互）
- 非功能需求（扩展性、可配置性、迭代速度）

AI 输出：2-3 套架构方案，每套包含：

- 模块划分逻辑（差异化的边界策略）
- 关键类的职责定义
- 方案优劣对比表

人工跟进：开发者评审各方案，选出最优方案或组合方案中的最佳部分。

节点 2：AI UML 图生成

输入：选定的架构方案描述 + 现有代码接口信息

AI 输出（PlantUML 格式）：

- 类图（Class Diagram）：Model/System/Command/Mono 类及关系
- 序列图（Sequence Diagram）：关键流程（关卡加载、对话推进、通关判定）

人工跟进：开发者审核图的完整性和正确性。根据 Ferrari 等人（2024）的研究，AI 生成的序列图在需求清晰时最接近人工质量，但需检查在需求歧义处的正确性[1]。

节点 3：AI 接口一致性校验

输入：PlantUML 类图中提取的接口签名 + 现有代码实际接口

AI 输出：

- 接口一致性报告（标注不匹配项）
- 耦合热点识别（如 `GameFlowController` 对各系统的依赖数）

3.4 工具链选型

工具	核心功能	在 P3 中的集成方式	预期效率
GPT-5.5	架构方案生成、模式推荐、接口一致性检查	通过结构化 Prompt 模板（Markdown 格式）输入约束，获取方案输出	方案探索量 ↑300%-500%
PlantUML + GPT-5.5	自然语言→UML 图自动生成	使用 Next-AI-PlantUML 或 text-to-diagram 工具链，从方案描述生成 .puml 文件，本地渲染	UML 绘制耗时 ↓30%-40%[2]
Codex	设计模式推荐、代码骨架生成	在编写架构入口类（如 <code>GameArchitecture</code> ）时，AI 实时推荐模式实现	骨架代码生成速度 ↑50%

3.5 AI 输出物质量标准与验证机制

AI 输出物	质量标准	验证方式
架构方案文档	至少 2 套差异化方案；每套含模块划分逻辑和优劣分析	开发者逐方案评审，填写《方案评审表》
PlantUML 类图	覆盖所有核心类；关系标注正确；无孤立节点	人工逐类核对 + PlantUML 语法编译通过
PlantUML 序列图	覆盖主路径和 2 条异常路径；参与者与类图一致	人工走查 + 与现有代码交互流程比对
接口一致性报告	列出所有不匹配项及严重级别	开发者逐项确认并标记修复/忽略

质量门禁：AI 输出的架构方案和 UML 图必须通过 **人工评审签字** 后方可进入 P4 实现阶段。不通过则迭代调整 Prompt 后重新生成。

3.6 预期效率提升

指标	改进前	改进后	提升幅度
架构方案探索数量	1-2 套	3-5 套	↑200%
UML 图绘制耗时	2-3 小时	1-1.5 小时	↓40%-50%

指标	改进前	改进后	提升幅度
接口不匹配发现时机	联调阶段	设计阶段	前置 2 个阶段

4. P4 实现与自测改进方案 (★ 重点)

4.1 当前过程回顾

P4 当前活动流程：

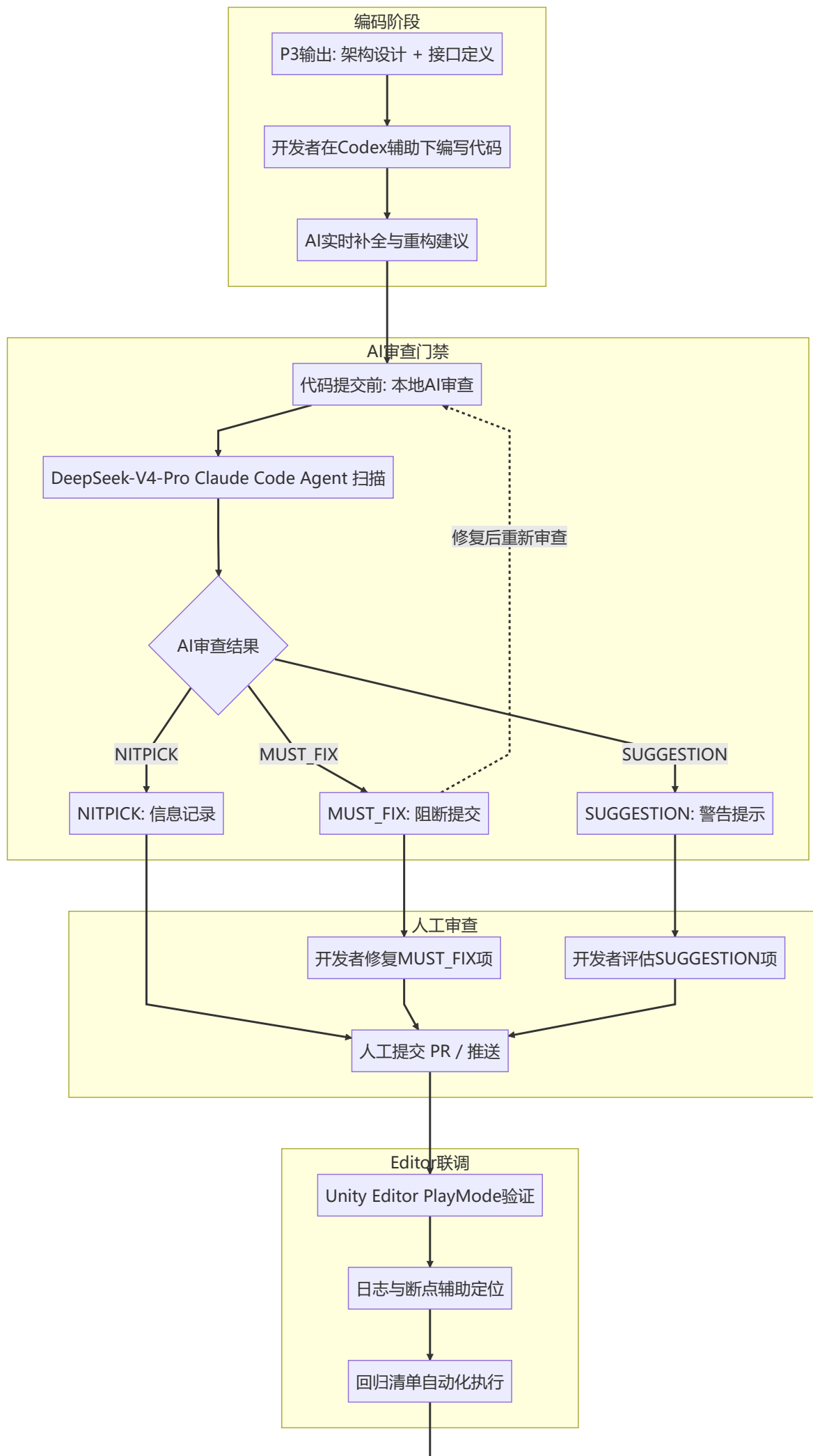


核心痛点：

- 人工代码审查覆盖不足（小团队无专职 Reviewer）
- Unity API 用法复杂（Addressables、UniTask），文档查阅耗时
- 缺陷发现滞后（联调阶段才发现编码阶段可拦截的问题）

4.2 AI 融入方案

核心思路：在代码提交与人工审查之间插入 AI 代码审查门禁，形成「AI 生成 → AI 审查 → 人工确认 → Editor 联调」四层流水线。



4.3 流程节点详解

节点 1：AI 辅助编码

工作方式：

- 开发者在 VS Code + Codex 中编写 C# 脚本
- AI 根据项目上下文（QFramework 架构、已有类的接口）实时补全代码
- 当检测到重复模式时，AI 主动提示重构建议

特别适用于：

- 模板化代码：实体类定义（Player/Box/Obstacle）、数据模型（LevelData/EntityData）
- Unity 胶水代码：`[SerializeField]` 声明、`GetComponent` 调用、`SceneManager` 操作
- UniTask 异步逻辑：关卡加载、资源预加载

注意事项：对核心流程逻辑（如 `GameFlowController.ExecuteStep()`）和性能敏感代码（如实体更新循环），AI 生成后须逐行审查。

节点 2：AI 代码审查门禁（关键改进）

工作方式：

- 开发者在提交代码前，运行 AI 审查工具扫描变更集
- 审查结果按严重级别分为三类：

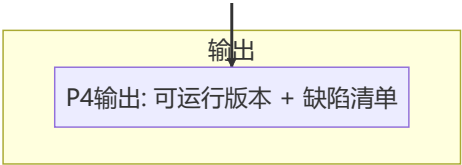
级别	说明	处理策略	Unity 项目示例
MUST_FIX	潜在运行时崩溃或逻辑错误	阻断提交，必须修复	空引用未检查、Addressables 资源未释放、async 方法未 await
SUGGESTION	代码异味或性能隐患	警告提醒，开发者评估	LINQ 在高频 Update 中、 <code>FindObjectsOfType</code> 在热路径中
NITPICK	风格或命名建议	仅供参考	变量命名约定、缺少 Summary 注释

审查维度：

- 空安全：检测可能的 `NullReferenceException`
- 资源管理：Addressables 加载/释放配对、`IDisposable` 实现
- 性能：主线程阻塞调用、频繁 GC 分配
- 架构一致性：是否遵守 Model/System/Command 分层约束
- Unity 最佳实践：`[RequireComponent]` 使用、`Awake` vs `Start` 时机

节点 3：人工确认与联调

- 开发者修复 MUST_FIX 项后重新提交审查
- SUGGESTION 项由开发者逐条评估，可接受或拒绝
- 通过门禁后进入 Unity Editor PlayMode 联调



- 回归清单由 AI 辅助生成并自动执行（详见场景四）

4.4 工具链选型

工具	核心功能	在 P4 中的集成方式	预期效率
Codex	C# 代码实时补全、上下文感知生成、重构建议	IDE 插件集成；支持项目级规则配置（如强制使用 QFramework 模式）	编码速度 ↑55.8% [3]
DeepSeek-V4-Pro + Claude Code Agent	CLI 驱动的深度代码审查、架构偏离检测	Agent 命令扫描变更集，输出 MUST_FIX/SUGGESTION/NITPICK 分级报告	审查覆盖率达 100% 代码量
DeepSeek-V4-Pro + Claude Code Agent	自动 PR 审查	配置审查规则，作为 Git 预提交 Hook 触发	PR 审查时间 ↓60%–75%[4]
Unity Test Framework + GPT-5.5	AI 辅助生成 PlayMode/EditMode 测试脚本	从 P4 的代码变更自动生成回归测试用例	测试编写耗时 ↓70%[5]

4.5 AI 输出物质量标准与验证机制

AI 输出物	质量标准	验证方式
AI 生成的代码	编译通过；无 MUST_FIX 级别审查问题；遵循项目命名约定	Unity Editor 编译 + AI 审查门禁 + 人工走查
AI 审查报告	覆盖所有变更文件；分级准确；无漏报崩溃级问题	人工抽查审查结果，校准分级
AI 重构建议	不改变外部行为；性能不劣化；通过所有现有回归用例	回归清单执行通过

质量门禁：MUST_FIX 级别问题清零为 **硬性门禁**，未通过不得进入 Editor 联调阶段。

4.6 预期效率提升

指标	改进前	改进后	提升幅度
模板化代码编写耗时	手动编写	AI 秒级生成	↓80%–90%
代码审查覆盖率	~30%（人工抽查）	100%（AI 全量扫描）	↑233%
缺陷发现阶段	联调阶段	编码阶段（AI 审查门禁）	前置 1–2 个阶段
Unity API 文档查阅频率	高频中断	AI 实时提示	↓60%

5. 其他过程改进简述

5.1 P2 需求与规则建模：AI 辅助需求捕获

改进方向：在 P2 的「提炼核心实体」活动前，增加 AI 辅助需求结构化节点——开发者将口头讨论的玩法点子输入 LLM，AI 输出结构化的实体-状态-约束文档和 JSON Schema 草稿。

工具：GPT-5.5 + 结构化 Prompt 模板（含实体定义、状态约束、关联关系）

预期效果：需求识别完整度提升（参考 Self-GenRE 方法论[6]），减少因口头传递导致的信息衰减。

5.2 P5/S2 测试与质量保证：AI 测试用例生成

改进方向：在 P5 回归验证前，由 AI 测试分析师自动生成测试用例，覆盖主路径、异常路径（关卡文件缺失、输入未配置、信号未触发）和边界条件。

工具：GPT-5.5 + Unity Test Framework（PlayMode/EditMode）

预期效果：测试用例编写耗时降低 70%-90%[5]，边界条件覆盖率系统化提升。

5.3 S3 文档与知识沉淀：AI 文档自动生成

改进方向：将 AI 文档生成融入 P3-P6 各过程的输出环节，由 AI 从代码和 Git 历史自动生成 API 文档、变更日志、开发约定更新。

工具：DeepSeek-V4-Pro + Claude Code Agent（文档生成模式）+ Git Diff 分析

预期效果：文档维护从「事后补课」变为「伴随生成」，API 文档 BLEU-4 得分约 43%[7]。

6. 工具链选型汇总矩阵

过程	AI 工具	核心功能	集成方式	预期提效	成熟度
P3 架构设计	GPT-5.5 + PlantUML	方案生成、UML 绘制、接口校验	Prompt 模板 + Next-AI-PlantUML	UML 耗时 ↓40%	★★★★
P4 编码	Codex	代码补全、上下文感知生成	IDE 集成	编码速度 ↑55%	★★★★★
P4 审查	DeepSeek-V4-Pro + Claude Code Agent	三级分级审查、门禁阻断	CLI + Agent 配置	审查覆盖 ↑233%	★★★★
P4 重构	Codex + DeepSeek-V4-Pro + Claude Code Agent	耦合识别、模式推荐、自动重构	IDE 集成 + CLI	重构效率 ↑50%	★★★★
P2 需求	GPT-5.5	需求结构化、Schema 生成	Prompt 模板	需求完整度 ↑29%	★★★
P5/S2 测试	LLM + Unity Test Framework	测试用例生成、缺陷预测	Prompt + Unity Test Runner	测试编写 ↓70%	★★★

过程	AI 工具	核心功能	集成方式	预期提效	成熟度
S3 文档	Claude Code + Git	文档、变更日志、约定维护	CLI + Git Hook	文档维护 ↓50%	★★★

7. 风险评估与应对策略

7.1 风险识别矩阵

编号	风险类别	描述	可能性	影响程度	风险等级
R1	数据安全风险	AI 工具将代码上传至云端处理，可能导致代码泄露；Prompt 中可能包含敏感配置信息	中	高	高
R2	AI 幻觉风险	LLM 生成不存在的 Unity API、错误的类名/方法签名、编造的引用文献	高	中	中高
R3	模型偏见风险	AI 生成代码偏向训练数据中的特定模式，可能在架构方案中忽略更优的非主流模式	中	中	中
R4	技术依赖风险	过度依赖 AI 工具导致开发者编码能力与架构判断力退化；AI 服务不可用时开发流程受阻	中	高	高
R5	集成复杂度风险	多 AI 工具链（Codex + DeepSeek-V4-Pro Claude Code Agent + PlantUML + GPT-5.5）的配置与维护成本超出预期	低	中	中低

7.2 应对策略

R1 数据安全风险

策略	具体措施
工具层	优先选择支持本地部署或明确承诺不将代码用于训练的 AI 服务（如 DeepSeek-V4-Pro 本地部署、GPT-5.5 API with Zero Data Retention）
流程层	建立「敏感信息剥离」规范：提交 AI 处理的代码前移除 API Key、内网地址等敏感配置
审查层	AI 审查门禁增加「敏感信息泄露检测」规则

R2 AI 幻觉风险

策略	具体措施
编译验证	所有 AI 生成代码必须通过 Unity Editor 编译（编译失败 = MUST_FIX 级别阻断）

策略	具体措施
API 背书	建立「AI 生成 API 白名单」——AI 建议的不常见 API 需开发者在 Unity 官方文档中确认后方可使用
双重检查	架构方案和 UML 图必须经人工评审签字，不对 AI 输出做「盲信」

R3 模型偏见风险

策略	具体措施
多方案强制生成	Prompt 中要求 AI 生成 ≥ 2 套差异化架构方案，防止单一模式偏好
人工评审把关	方案评审表中设「方案多样性」评价项，由开发者判断是否遗漏重要方案
跨模型交叉验证	关键架构决策使用至少 2 个不同模型（如 GPT-5.5 + DeepSeek-V4-Pro）独立生成建议并对比

R4 技术依赖风险

策略	具体措施
无 AI 日	每周安排半个工作日完全脱离 AI 工具编码，保持原生开发能力
AI 输出 « 必须理解 » 原则	开发者对 AI 生成的每行代码必须理解其意图，禁止粘贴不明代码
渐进式自动化	先从审查和文档等辅助环节引入 AI，核心架构和逻辑设计保持人工主导
应急预案	当 AI 服务不可用时，回退至传统人工流程——原有过程文档作为备份流程参考

R5 集成复杂度风险

策略	具体措施
渐进式引入	第一阶段只引入 Codex + DeepSeek-V4-Pro Claude Code Agent；第二阶段再引入 PlantUML 工具链和 GPT-5.5
配置即代码	所有 AI 工具配置（ <code>.cursorrules</code> 、 <code>.coderabbit.yaml</code> 、Prompt 模板）纳入 Git 版本管理
月度工具评审	每月评估 AI 工具链的投资回报率，裁撤低效工具

8. 实施路线图

阶段	周期	内容	关键产出
第一阶段	第 1–2 周	P4 编码辅助引入：部署 Codex + DeepSeek-V4-Pro Claude Code Agent 审查门禁	审查规则配置、AI 审查标准文档
第二阶段	第 2–3 周	P3 架构辅助引入：建立 Prompt 模板库 + PlantUML 工具链	P3 Prompt 模板、PlantUML 图模板
第三阶段	第 3–4 周	扩展覆盖：P2 需求、P5/S2 测试、S3 文档的 AI 辅助引入	全流程改进后过程定义文档

参考文献

[1] Ferrari, A. et al. (2024). Model Generation with LLMs: From Requirements to UML Sequence Diagrams. *IEEE REW 2024 / MoDRE Workshop*.

-  arXiv: <https://arxiv.org/abs/2404.06371>
-  Google Scholar: <https://scholar.google.com/scholar?q=Model+Generation+LLMs+Requirements+UML+Sequence+Diagrams+Ferrari+2024>

[2] Tagliaferro, A., Corbo, S. & Guindani, B. (2025). Leveraging LLMs to Automate Software Architecture Design from Informal Specifications. *IEEE ICSCA-C 2025*, pp. 291–299.

-  DOI: <https://doi.org/10.1109/ICSCA-C65153.2025.00049>
-  Google Scholar: <https://scholar.google.com/scholar?q=Leveraging+LLMs+Automate+Software+Architecture+Design+Informal+Specifications>

[3] Science China Information Sciences. (2025). A Survey on Large Language Models for Software Engineering.

-  DOI: <https://doi.org/10.1007/s11432-025-4670-0>
-  Google Scholar: <https://scholar.google.com/scholar?q=survey+large+language+models+software+engineering+2025>

[4] DeepSeek-V4-Pro + Claude Code Agent. AI-Powered Code Review Pipeline. (2025).

[5] Celik, A. & Mahmoud, Q. (2025). A Review of Large Language Models for Automated Test Case Generation. *MAKE*, MDPI, 7(3), 97.

-  DOI: <https://doi.org/10.3390/make7030097>
-  Google Scholar: <https://scholar.google.com/scholar?q=Review+Large+Language+Models+Automated+Test+Case+Generation+Celik+2025>

[6] Cheng, H. et al. (2025). Generative AI for Requirements Engineering: A Systematic Literature Review. *Software: Practice and Experience*, Wiley.

-  arXiv: <https://arxiv.org/abs/2409.06741>
-  Google Scholar: <https://scholar.google.com/scholar?q=Generative+AI+Requirements+Engineering+Systematic+Literature+Review+Cheng+2024>

[7] Venigalla, A.S.M. & Chimalakonda, S. (2025). DocFetch — Towards Generating Software Documentation from Multiple Software Artifacts. *arXiv:2508.17719*.

-  arXiv: <https://arxiv.org/abs/2508.17719>
 -  Google Scholar: <https://scholar.google.com/scholar?q=DocFetch+Generating+Software+Documentation+Multiple+Artifacts+2025>
-

撰写日期: 2026 年 6 月 13 日